SQL Query Review Report

Query Review 1

Original Query

```
SELECT e.first_name, e.last_name, d.department_name
FROM employees e
JOIN departments d
ON e.employee_id = d.department_id;
```

Analysis

The query is syntactically valid but the JOIN condition is incorrect.

`EMPLOYEE_ID` identifies an employee, while `DEPARTMENT_ID` identifies a department. These columns represent different business entities and are not related through a foreign key relationship.

The correct relationship is:

`EMPLOYEES.DEPARTMENT_ID → DEPARTMENTS.DEPARTMENT_ID`

Using `EMPLOYEE_ID = DEPARTMENT_ID` may return incorrect or incomplete results because it matches unrelated values.

Corrected Query

```
SELECT e.first_name,
       e.last_name,
       d.department_name
FROM employees e
INNER JOIN departments d
       ON e.department_id = d.department_id;
```

Reason for Improvement

The revised query follows the actual foreign key relationship between employees and departments, ensuring accurate business reporting.

Query Review 2

Original Query

```
SELECT e.first_name, e.last_name, j.job_title
FROM employees e
JOIN jobs j
ON e.employee_id = j.job_id;
```

Analysis

The query is syntactically valid but the JOIN condition is incorrect.

`EMPLOYEE_ID` and `JOB_ID` are unrelated attributes. An employee is linked to a job through the `JOB_ID` column.

The correct relationship is:

`EMPLOYEES.JOB_ID → JOBS.JOB_ID`

Corrected Query

```
SELECT e.first_name,
       e.last_name,
       j.job_title
FROM employees e
INNER JOIN jobs j
       ON e.job_id = j.job_id;
```

Reason for Improvement

The revised query correctly connects employees to their assigned jobs and returns accurate job information.

Query Review 3

Original Query

```
SELECT e.first_name, d.department_name
FROM employees e
LEFT JOIN departments d
ON e.department_id = d.department_id
WHERE d.department_name = 'Sales';
```

Analysis

The query contains a business logic issue.

A LEFT JOIN is intended to return all employees, including those without departments.

However, the WHERE clause:

```
WHERE d.department_name = 'Sales'
```

filters out all NULL department records, effectively converting the LEFT JOIN into an INNER JOIN.

Corrected Query

If the objective is to retrieve employees who belong to the Sales department:

```
SELECT e.first_name,
       d.department_name
FROM employees e
INNER JOIN departments d
       ON e.department_id = d.department_id
WHERE d.department_name = 'Sales';
```

Reason for Improvement

Since only Sales employees are required, an INNER JOIN is more efficient and clearly expresses the business requirement.

Query Review 4

Original Query

```
SELECT e.first_name, m.first_name AS manager_name
FROM employees e
JOIN employees m
ON e.employee_id = m.manager_id;
```

Analysis

This is a self-join, but the relationship direction is reversed.

The business requirement is:

"Show each employee and their manager."

The correct relationship is:

```
EMPLOYEES.MANAGER_ID → EMPLOYEES.EMPLOYEE_ID
```

The original query returns managers and their subordinates instead of employees and their managers.

Corrected Query

```
SELECT e.first_name,
       m.first_name AS manager_name
FROM employees e
LEFT JOIN employees m
       ON e.manager_id = m.employee_id;
```

Reason for Improvement

The revised query correctly maps each employee to their direct manager.

A LEFT JOIN is preferred because top-level managers may not have a manager themselves, and they should still appear in the report.